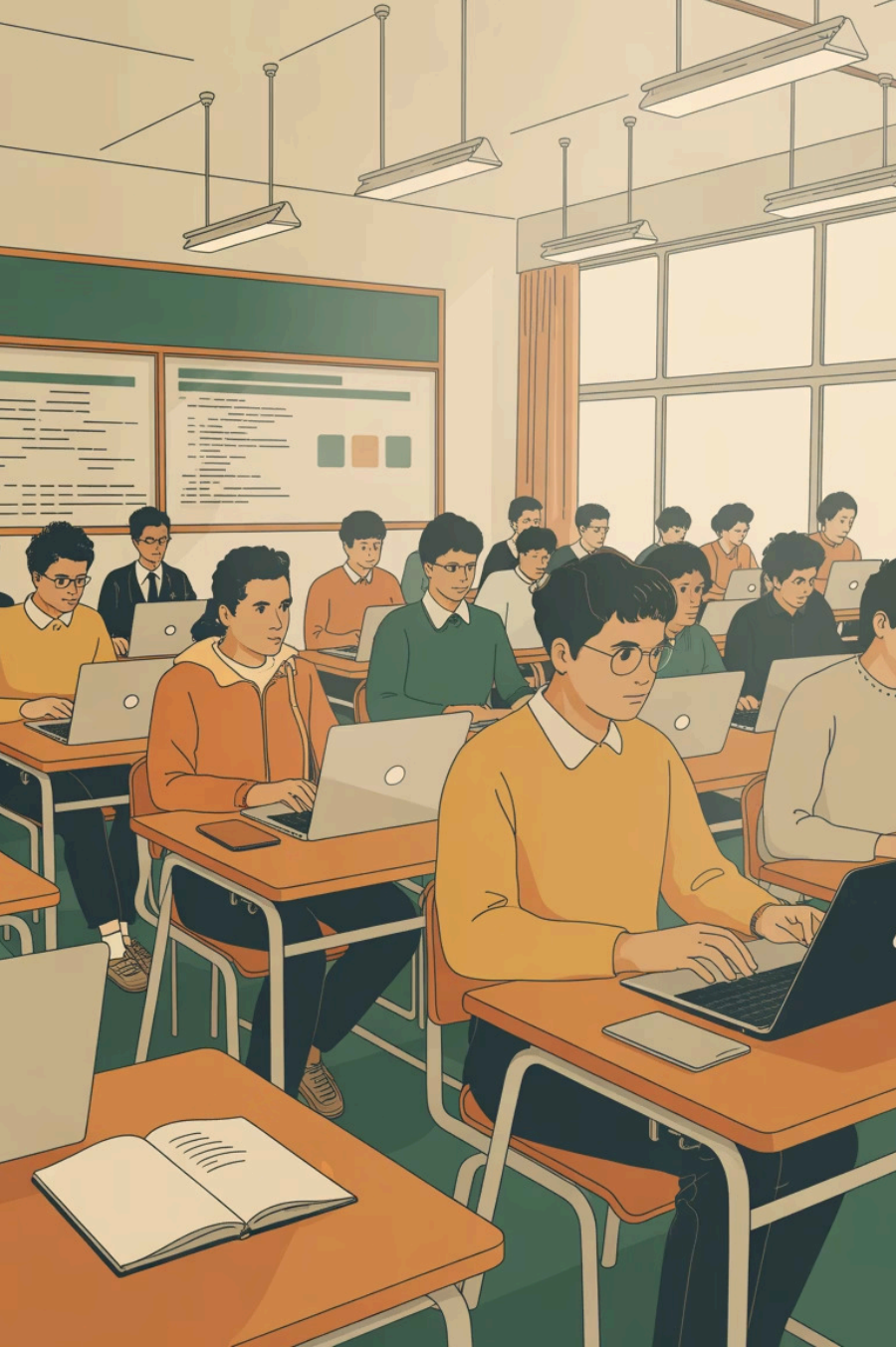




ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

Frameworks Backend

Semana 2 — Aula Prática

Tema: Preparação do Ambiente e Primeiro Contato com Framework

Carga horária: 4 horas

Prof. Ysmael Marinho Santos Júnior

Objetivos da Aula

1

Configurar Ambiente Backend

Instalar e verificar Node.js, Java e IDEs recomendadas para o laboratório.

2

Entender o Papel do Runtime

Compreender o que é runtime, servidor HTTP e API REST na prática.

3

Criar API sem Framework

Desenvolver um servidor HTTP básico usando apenas os recursos nativos da linguagem.

4

Criar API com Framework

Reproduzir a mesma API utilizando Express (Node.js) e Spring Boot (Java).

5

Comparar Abordagens

Analisar verbosidade, produtividade e organização entre as abordagens.

O que você precisa saber antes de codar

Runtime

Ambiente de execução de um programa. O **runtime** fornece os recursos necessários para que o código rode: gerenciamento de memória, acesso ao sistema operacional e APIs nativas.

Exemplos: **Node.js** (runtime JS) e **JVM** (runtime Java).

Servidor HTTP

Processo que fica **escutando** uma porta de rede, aguardando requisições HTTP. Ao receber uma requisição, processa e devolve uma resposta com status, cabeçalhos e corpo.

API REST

Interface que segue os princípios REST: uso de **verbos HTTP** (GET, POST, PUT, DELETE), recursos identificados por **URIs** e troca de dados em **JSON/XML**.

Framework

Estrutura pré-construída que fornece componentes, padrões e fluxo de execução. O framework **inverte o controle**: ele chama o seu código, não o contrário.

 **Biblioteca vs. Framework:** Na biblioteca, **você chama o código dela**. No framework, **ele chama o seu código** — princípio da Inversão de Controle (IoC).

Preparação do Ambiente

Node.js

Acesse **nodejs.org** e baixe a versão LTS.

```
$ node -v  
v20.x.x
```

```
$ npm -v  
10.x.x
```

Verifique se os dois comandos retornam versões sem erro.

Java (JDK 17+)

Baixe o **JDK 17 LTS** em adoptium.net ou via gerenciador de pacotes.

```
$ java -version  
openjdk 17.x.x
```

```
$ javac -version  
javac 17.x.x
```

IDEs Recomendadas

VS Code

Para Node.js / Express.
Instale as extensões:
REST Client, **ESLint** e
Node.js Extension Pack.

IntelliJ IDEA / Eclipse

Para Java / Spring Boot. O
IntelliJ Community já
inclui suporte nativo ao
Spring.



⚠ Dica: Se o comando não for reconhecido, verifique se o caminho de instalação está na variável de ambiente `PATH`.

Servidor HTTP Nativo com http Module

O módulo `http` é nativo do Node.js — nenhuma instalação adicional necessária. Crie o arquivo `server.js`:

```
const http = require('http');

const server = http.createServer((req, res) => {
  if (req.url === '/hello' && req.method === 'GET') {
    res.writeHead(200, { 'Content-Type': 'application/json' });
    res.end(JSON.stringify({ message: 'Hello, World!' }));
  } else {
    res.writeHead(404);
    res.end(JSON.stringify({ error: 'Not found' }));
  }
});

server.listen(3000, () => {
  console.log('Servidor rodando em http://localhost:3000');
});
```

Execute com: `node server.js` e acesse `http://localhost:3000/hello`

⚠ Problemas desta Abordagem



Escalabilidade

Múltiplas rotas exigem `if/else` encadeados.
Rapidamente se torna inviável.



Organização

Sem separação de responsabilidades. Tudo em um único arquivo monolítico.



Tratamento de Erros

Sem middleware de erro global. Cada rota precisa de tratamento manual.

Express.js: Simplificando o Desenvolvimento

Instale o Express via **npm** e crie o arquivo `app.js`:

```
# Inicializar projeto e instalar Express
$ npm init -y
$ npm install express
```

```
const express = require('express');
const app = express();

app.use(express.json());

app.get('/hello', (req, res) => {
  res.json({ message: 'Hello, World!' });
});

app.listen(3000, () => {
  console.log('Servidor Express em http://localhost:3000');
});
```

Execute com `node app.js` — comportamento idêntico ao anterior, com muito menos código!

Node Puro vs Express

Aspecto	Node Puro	Express
Linhas de código	~15	~8
Roteamento	Manual (if/else)	<code>app.get/post/</code> ...
Middleware	Nenhum	Integrado
JSON parsing	Manual	<code>express.json()</code>
Erros globais	Não	Sim
Escalabilidade	Baixa	Alta

HttpServer Nativo da JDK

O Java possui a classe `com.sun.net.httpserver.HttpServer` para criação de servidores sem dependências externas:

```
import com.sun.net.httpserver.*;
import java.io.*;
import java.net.*;

public class Main {
    public static void main(String[] args) throws Exception {
        HttpServer server =
            HttpServer.create(new InetSocketAddress(8080), 0);

        server.createContext("/hello", exchange -> {
            String response = "{\"message\":\"Hello, World!\"}";
            exchange.getResponseHeaders()
                .set("Content-Type", "application/json");
            exchange.sendResponseHeaders(200,
                response.length());
            OutputStream os = exchange.getResponseBody();
            os.write(response.getBytes());
            os.close();
        });

        server.start();
        System.out.println("Servidor em http://localhost:8080");
    }
}
```

Problemas do Java Puro

Alto Boilerplate

Muitas linhas de código para uma única rota simples. Gerenciamento manual de streams, cabeçalhos e status codes.

Sem Injeção de Dependência

Nenhum mecanismo nativo para gerenciar dependências entre componentes da aplicação.

Sem Serialização Automática

JSON precisa ser construído como `String` manualmente — propenso a erros.

 **Reflexão:** Imagine 20 endpoints neste padrão. O código se tornaria inviável de manter sem uma estrutura padronizada.

Spring Boot: Produtividade ao Máximo

Crie o projeto em **start.spring.io** (Spring Initializr) com a dependência **Spring Web**, depois crie seu controller:

```
@RestController
public class HelloController {

    @GetMapping("/hello")
    public Map<String, String> hello() {
        return Map.of("message", "Hello, World!");
    }

}
```

Execute com `./mvnw spring-boot:run` — o servidor sobe automaticamente na porta `8080`.

Java Puro vs Spring Boot

Aspecto	Java Puro	Spring Boot
Linhas de código	~25	~5
Roteamento	createContext ()	@GetMapping
JSON automático	Não	Sim (Jackson)
Injeção de dep.	Não	Sim (IoC)
Config. servidor	Manual	Automática
Escalabilidade	Baixa	Alta

Node Puro vs Express vs Java Puro vs Spring Boot

Critério	Node Puro	Express	Java Puro	Spring Boot
Verbosidade	Média	Baixa	Alta	Baixa
Produtividade	Baixa	Alta	Baixa	Alta
Escalabilidade	Baixa	Alta	Baixa	Muito Alta
Organização	Manual	Boa	Manual	Excelente
Curva de aprendizado	Baixa	Baixa	Média	Alta



Conclusão: Frameworks reduzem drasticamente o **boilerplate**, padronizam a arquitetura e aumentam a produtividade — ao custo de uma curva de aprendizado inicial.



Desafio em Sala

O Desafio

Implemente a rota `/status` em **pelo menos um** dos ambientes abaixo, retornando o JSON:

```
{
  "status": "ok",
  "framework": "nome-do-framework"
}
```

A rota deve responder ao método **GET** na porta padrão do ambiente escolhido.

Organização dos Grupos

Grupo A — Node Puro

Implemente usando apenas o módulo `http` nativo do Node.js. Sem `npm install`.

Grupo B — Express

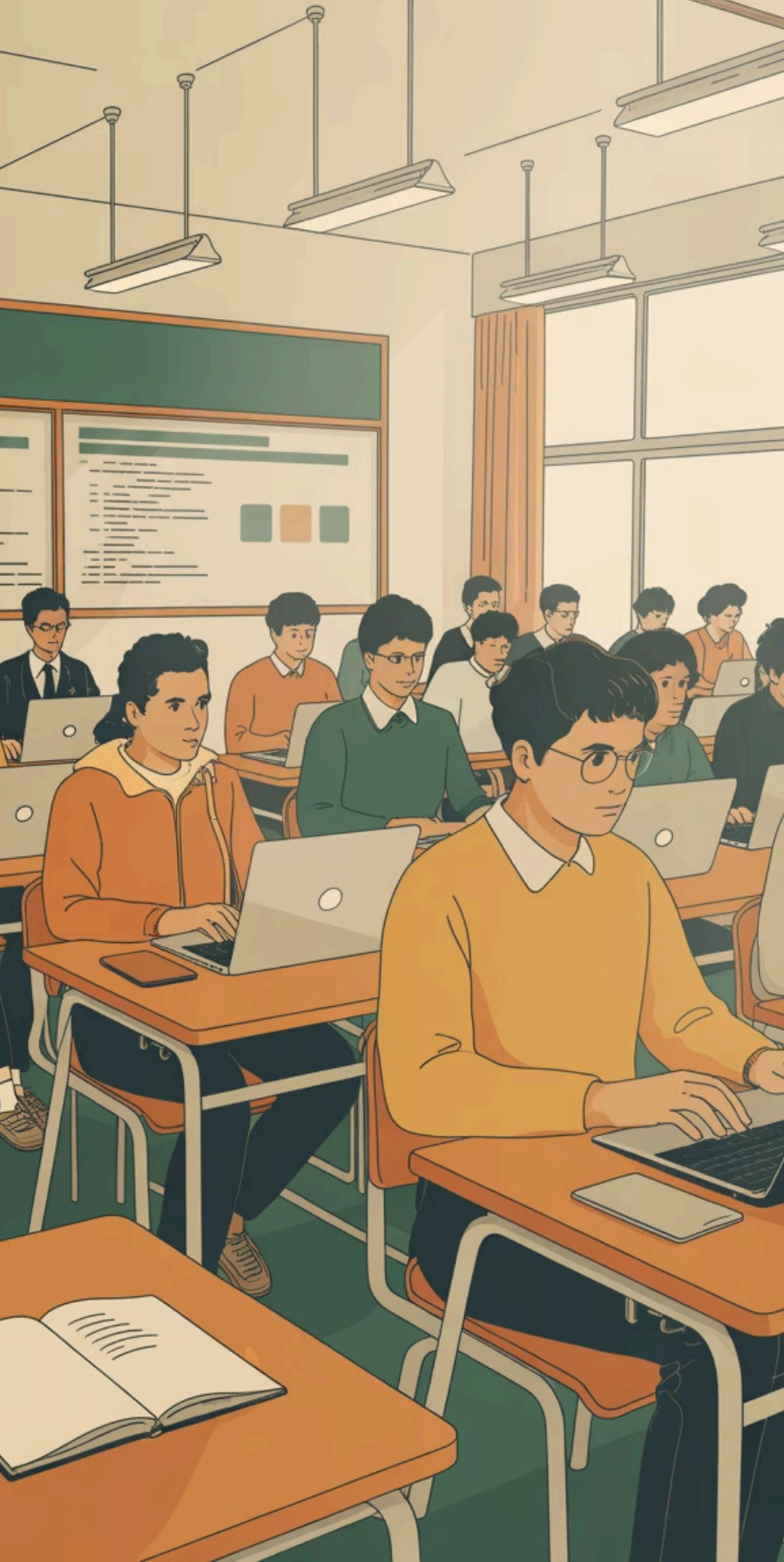
Implemente usando `express`. Lembre-se de inicializar o projeto com `npm init`.

Grupo C — Spring Boot

Crie um novo projeto no Spring Initializr e implemente usando `@RestController`.



Bônus: Adicione o campo `"timestamp"` com a hora atual na resposta JSON!



Encerramento

Reflexão Final

🤔 O que o framework realmente resolve?

Frameworks eliminam o **trabalho repetitivo e já resolvido**, permitindo que você foque na lógica de negócio. Eles não resolvem problemas de design — apenas os amplificam.

🚫 Quando não usar um framework?

Scripts simples, microserviços minimalistas, contextos de aprendizado ou situações onde o **overhead supera o benefício**. Simplicidade tem valor.

? Framework é obrigatório?

Não. É uma escolha técnica. O profissional sênior sabe *quando usar* e *quando não usar* — e entende os fundamentos mesmo sem a ajuda do framework.

Próxima aula: Aprofundamento em roteamento, middlewares e estrutura de projetos com Express e Spring Boot.

BONS ESTUDOS! 🚀